

Fine-Tuning LLMs

Adapting pre-trained models to specific tasks and domains

1. What is Fine-Tuning?

A base LLM is trained on enormous amounts of general text from the internet. It is good at many things, but it does not know your company's writing style, your domain's terminology, your product's specifics, or how to behave according to your exact requirements.

Fine-tuning is the process of taking that pre-trained model and continuing to train it on a smaller, curated dataset that is specific to your use case. The model adapts — it learns to respond in a particular style, follow specific formats, handle domain-specific language, or align with your guidelines.

Simple version: Pre-training = the model learns general language from billions of documents. Fine-tuning = you teach it your specific job using hundreds or thousands of your own examples.

Fine-tuning vs RAG vs Prompting — when to pick what

Approach	What it changes	Cost	Best when...
Prompting	Nothing — guides behavior via instructions	Zero	Task is simple and well-defined in a prompt
RAG	Gives model access to new knowledge	Low	Model needs up-to-date or private facts
Fine-tuning	Changes the model's weights	Medium-High	Style, tone, format, or domain language must change
Full pre-training	Trains from scratch	Very High	Completely new domain with no existing base model

Rule of thumb: Try prompting first. If that fails, try RAG. If the model still gets tone, format, or domain language wrong, that is when fine-tuning makes sense.

2. Fine-Tuning Methods

Full Fine-Tuning

All of the model's weights are updated during training. The entire model learns from your data.

- Most powerful — the model can change deeply
- Most expensive — requires many GPUs and a lot of memory (a 7B model needs ~100GB+ VRAM for full FT)
- Risk of catastrophic forgetting — the model can lose its general capabilities while learning your specific task
- Rarely used in practice today unless you have very specific needs and serious compute budget

SFT — Supervised Fine-Tuning

You provide labeled examples in a (prompt, ideal response) format and train the model to reproduce those responses. This is the most common starting point for fine-tuning.

- Dataset format: each example is an instruction + ideal completion pair

- The model learns to follow the style, format, and content of your labeled examples
- Works well for: consistent output formats, domain-specific responses, task-specific behavior
- ~1,000 high-quality examples are often enough for noticeable improvement

LoRA — Low-Rank Adaptation

LoRA is the most widely used fine-tuning method in 2025-2026. Instead of updating all the model's weights, LoRA freezes the base model and inserts small trainable matrices (called adapters) into each transformer layer.

- Only ~0.1% of the model's parameters are actually trained
- Uses ~70% less GPU memory than full fine-tuning
- Output quality is near-identical to full fine-tuning for most tasks
- The adapter is a tiny file (10-100 MB) that sits on top of the frozen base model
- You can swap adapters at serving time — one base model, many specialized adapters

Why LoRA dominates: You can fine-tune a 7B model on a single A100 GPU in an hour for under \$10. The adapter file is small enough to version-control and ship easily.

QLoRA — Quantized LoRA

QLoRA combines LoRA with 4-bit quantization of the base model. The base model is compressed to use 4 bits per weight instead of 16 or 32, which cuts memory usage dramatically.

- Lets you fine-tune a 70B model on a single 48GB GPU
- Tiny quality loss from quantization, huge memory saving
- The go-to method when GPU memory is the constraint
- Used with the Hugging Face PEFT + bitsandbytes libraries

RLHF — Reinforcement Learning from Human Feedback

RLHF is the technique behind ChatGPT, Claude, and Gemini's alignment. It teaches the model not just to complete text but to produce responses that humans actually prefer.

The RLHF pipeline has three stages:

- **Stage 1 — SFT:** Start with supervised fine-tuning on high-quality demonstrations
- **Stage 2 — Reward model:** Human annotators rank multiple model responses for the same prompt. A separate model (the reward model) is trained to predict which responses humans prefer.
- **Stage 3 — RL optimization:** The main model is trained using PPO (Proximal Policy Optimization) to generate responses that score highly according to the reward model.

RLHF is expensive and complex. It requires large amounts of human preference data and careful tuning of the RL training loop.

DPO — Direct Preference Optimization

DPO is a simpler, more stable alternative to RLHF that has largely replaced it in production teams by 2026. Instead of training a separate reward model and running RL, DPO directly trains the model on preference pairs — examples where you have a chosen (good) response and a rejected (bad) response for the same prompt.

- No reward model needed — just pairs of (prompt, chosen response, rejected response)
- No unstable RL training — just standard supervised training
- Results are comparable to RLHF with much less complexity
- Dataset format: {'prompt': '...', 'chosen': '...', 'rejected': '...'}

Method	What it trains	GPU need	Complexity	Best For
Full fine-tuning	All weights	Very High	Low	Deep domain adaptation, big budget
SFT	All weights (task-specific)	High	Low	Teaching output format and style
LoRA	Small adapter matrices only	Low	Low	Most fine-tuning tasks — default choice
QLoRA	LoRA on quantized model	Very Low	Low	Large models on limited GPU memory
RLHF	Alignment via human reward	Very High	Very High	Alignment, safety, preference learning
DPO	Preferences directly	Medium	Medium	Alignment without RL complexity

3. Data Preparation

Data quality is the single biggest factor in fine-tuning success. A model trained on 500 clean, well-labeled examples will outperform one trained on 10,000 messy ones.

Dataset format for SFT (most common)

Each row in your dataset should be a prompt-completion pair. The exact format depends on the model but typically looks like this:

- Input (prompt): the instruction or question the model will receive
- Output (completion): the exact response you want the model to produce
- Most frameworks use the chat template format: system message + user message + assistant message

How much data do you need?

Goal	Approx. examples needed
Learn a specific output format (e.g. always return JSON)	100 – 500
Adopt a writing style or tone	500 – 1,000
Domain-specific Q&A (medical, legal, finance)	1,000 – 5,000
Multi-task instruction following	5,000 – 50,000
RLHF / DPO preference alignment	10,000+ preference pairs

Data quality checklist

- Remove duplicates — identical or near-identical examples cause overfitting
- Check for label consistency — if two similar prompts have different ideal responses, the model gets confused
- Make outputs diverse — if all examples have the same structure, the model becomes rigid
- Include edge cases — examples that represent tricky or unusual inputs improve robustness
- Split your data: 80% train, 10% validation, 10% test. Never evaluate on training data.

4. Training Process — Key Hyperparameters

You do not need to deeply understand every hyperparameter, but these are the ones that come up in interviews and production work:

Parameter	What it controls	Typical starting value
Learning rate	How big each weight update step is. Too high = training diverges. Too low = slow or no learning.	1e-4 to 2e-5 for SFT
Epochs	How many times the model sees the whole dataset.	1–3 (more risks overfitting)
Batch size	Number of examples processed per step. Larger = more stable gradients, more memory.	4–32 depending on GPU
LoRA rank (r)	Size of the adapter matrices. Higher r = more capacity but more memory.	8–64 (16 is a good default)
LoRA alpha	Scaling factor for the adapter. Often set to 2x the rank.	16–128
Max sequence length	Maximum token length per example. Longer = more memory.	512–4096

Watch for overfitting: If training loss keeps falling but validation loss starts rising, the model is memorizing examples rather than learning. Use early stopping and reduce epochs.

5. Evaluation

After fine-tuning, you need to verify the model actually improved — and that it did not get worse at things it was already good at.

Automated metrics

- **BLEU / ROUGE:** Measures overlap between model output and reference text. Useful for summarization and translation.
- **Perplexity:** How surprised the model is by the validation data. Lower is better — it means the model finds your examples predictable (i.e. it has learned the pattern).
- **Task-specific accuracy:** For classification tasks, just measure correct vs incorrect on a held-out test set.

LLM-as-judge evaluation

Use a strong model (GPT-4o, Claude Opus) to score your fine-tuned model's outputs on dimensions like relevance, accuracy, tone, and format. This is now a standard production practice and scales much better than human evaluation.

Regression testing

Make sure fine-tuning on your specific task has not broken the model's general abilities. Always run your fine-tuned model on a set of general-purpose benchmarks alongside your task-specific ones. If general performance drops significantly, the model has catastrophically forgotten — reduce epochs or learning rate and retrain.

6. Serving Fine-Tuned Models

- **LoRA adapters are tiny** — 0-100 MB) and can be hot-swapped on a base model at serving time — one GPU, multiple specialized adapters loaded on demand
- **vLLM supports LoRA serving** — tively: load the base model once, attach the right adapter per request based on tenant or use case
- **For full fine-tuned models** — erge the adapter into the base model weights using `merge_and_unload()` in PEFT, then deploy the merged model as you would any other
- **Version your adapters** — e same way you version code — each fine-tune run produces a new adapter version in your model registry

7. Quick Reference

Question	Answer
Should I fine-tune or use RAG?	RAG for knowledge gaps. Fine-tune for style, format, tone, domain language.
What method should I start with?	LoRA + SFT. It is cheap, fast, and works for most tasks.
How many examples do I need?	500–1,000 high-quality examples is a solid starting point.
How do I prevent overfitting?	Use early stopping, keep epochs low (1–3), validate on held-out data.
How do I align model behavior?	Use DPO with chosen/rejected response pairs. Simpler than RLHF.
What is LoRA rank?	Size of adapter matrices. Start at 16. Increase if model under-fits.
How do I serve multiple fine-tunes cheaply?	One base model + hot-swappable LoRA adapters in vLLM.
What is catastrophic forgetting?	Model loses general abilities after fine-tuning. Fix: reduce epochs, use LoRA not full FT.